

OBSERVER news-alpha

Documento di Origine del Progetto

Reverse engineering 'intelligente' a partire dal codice e dai canonici .doc

Data: 20 gennaio 2026

Scopo del documento

Spiegare in modo chiaro (anche a non tecnici) che cos'è OBSERVER news-alpha, come crea valore oggi, come sono collegati i suoi moduli, e come da questa descrizione discendono i documenti canonici della cartella **.doc**.

Nota: questo documento è concepito come testo 'origine'. I documenti in .doc sono una scomposizione operativa/tecnica derivata, e possono evolvere in struttura senza cambiare la sostanza qui descritta.

Indice

Placeholder for table of contents	0
-----------------------------------	---

1. In una frase

OBSERVER news-alpha è un sistema locale (offline-by-default) che trasforma dati di mercato e notizie in un insieme di **decisioni osservabili**: cosa sarebbe sensato fare oggi (as-of) e cosa sarebbe successo nel passato (audit/backtest), con un 'scontrino' completo che spiega ogni passaggio.

2. Il problema che risolve

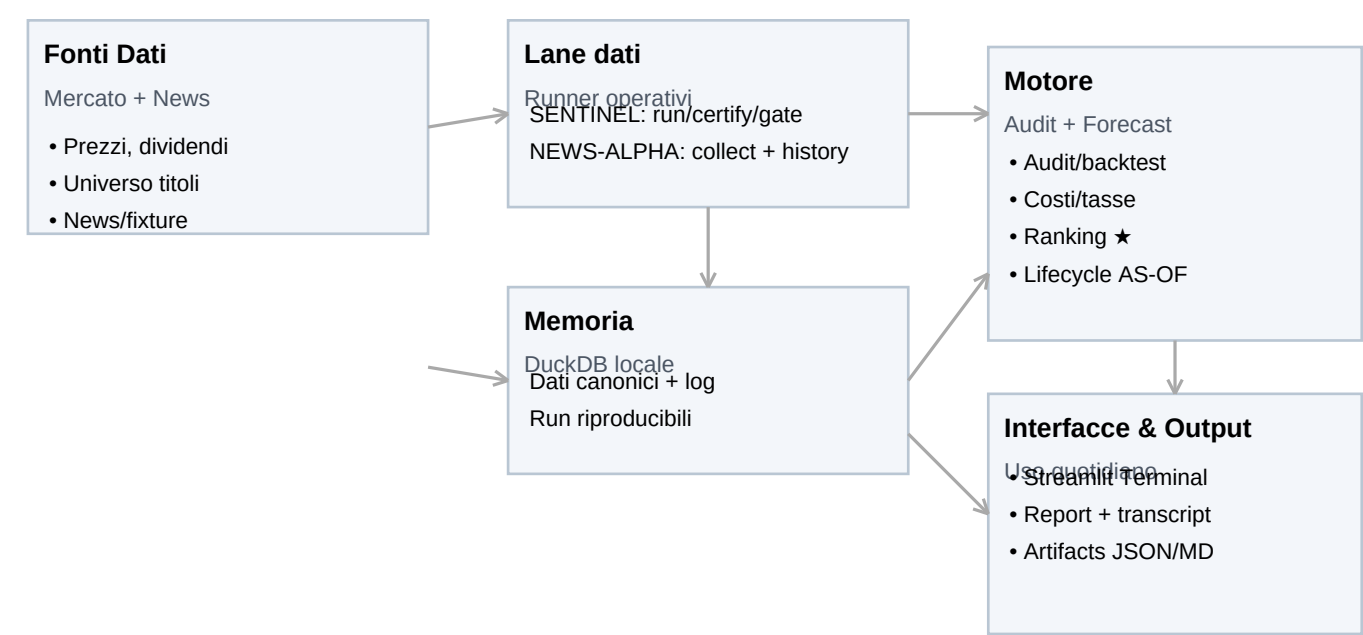
Nel lavoro quotidiano su mercati e news, il rischio più comune non è non avere un'idea, ma **non sapere se ci si può fidare** dell'idea: dati incompleti, look-ahead involontario, ticker sporchi, buchi di calendario, metriche che sembrano buone solo perché calcolate male. OBSERVER nasce per ridurre questi auto-inganni.

3. Il valore oggi e il potenziale domani

- **Valore oggi:** pipeline riproducibile che produce ranking/forecast e audit report, e segnala apertamente quando qualcosa non è coperto (data gaps, forced exits, provenance mancante).
- **Valore oggi:** UI locale che separa chiaramente la vista decisionale (Trading Room AS-OF) dalla vista storica (audit/backtest).
- **Potenziale:** delivery/alert, contratti news più raffinati e (se mai voluto) un layer di esecuzione; senza perdere auditabilità.

4. L'applicazione come organismo (vista a moduli)

Per evitare che il progetto appaia come una lista di feature scollegate, lo descriviamo come un organismo: ogni modulo ha un ruolo preciso e scambia dati e segnali con gli altri.



Organo (metafora)	Nome nel progetto	Responsabilit� (in parole semplici)
Nervi / Comando	SENTINEL (scripts/sentinel.py)	Esegue pipeline end-to-end, applica gate, produce run_id e transcript
Sensi (News)	NEWS-ALPHA (scripts/news_alpha.py) e news_alpha.py	Raccolgono news in modo deterministico: collector e history lane (GDE)
Memoria	DuckDB locale (data/sentinel_alpha.db)	Conserva dati e risultati: prezzi, segnali, run, report.
Cervello	Audit Engine + Forecast (src/core/*_simulazione.py)	Simulazione: timing T+1, costi/tasse, ranking a stelle, lifecycle.
Occhi	Streamlit Terminal (app.py + pages/)	Giuscotto: Decision Briefing, gates, audit runs, trades/equity, lifecycle
Diario clinico	reports/*	Report e artefatti leggibili (MD/JSON) per capire il perche dei risultati

5. Fasi operative: cosa succede in pratica

Il sistema e progettato per essere usato per fasi. Ogni fase produce output e controlli che rendono il passaggio successivo piu sicuro.

Fasi operative tipiche



Nota: ogni fase produce tracce (DB + report) per evitare risultati 'magici' non spiegabili.

Scenario	Domanda dell'utente	Cosa fa OBSERVER	Output principale
Decisione quotidiana	Oggi cosa e tradabile e perche	Carica ranking e mostra motivazioni e stato	Decision Briefing + Lifecycle
Audit periodico	Nel passato, il metodo e robusto	Segue audit/backtest con regole conservative	AUDIT_COMPLETE.md + ta
Triage dati	Perche il risultato non esce / e	Mostra gate e problemi: prezzi mancanti, tick	GateSpideTriagedata
Ricerca news storica	Cosa c'era in un giorno preciso	History lane GDELT: download idempotente, profiles fixtures	news JSONL

6. Principi di progettazione (il 'perche' delle scelte)

Questi principi sono il nucleo del progetto. Sono piu importanti delle singole feature, perche guidano come il sistema cresce senza perdere credibilita.

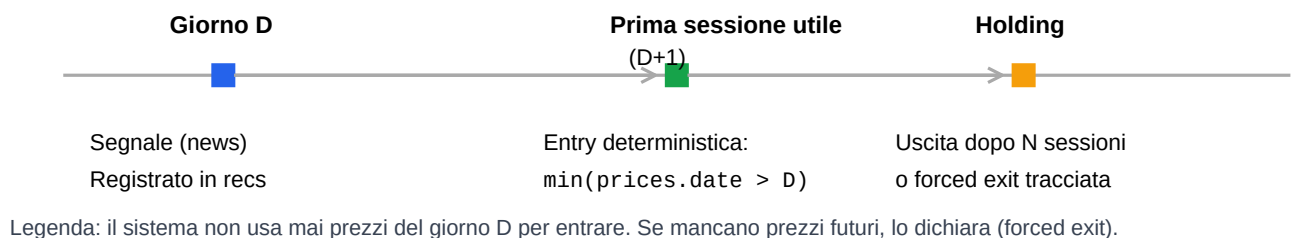
6.1 Offline-by-default

Di default il sistema usa solo dati locali. Le azioni online (es. download news o backfill prezzi) sono sempre esplicite e guardate da flag/variabili d'ambiente. Questo riduce sorprese e rende i risultati ripetibili.

6.2 No future data (niente look-ahead involontario)

Un segnale del giorno D non puo entrare a prezzi del giorno D. L'ingresso e sempre alla prima sessione utile successiva (T+1).

Contratto temporale (No Future Data): T+1



6.3 Auditabilita e disclosure

Ogni run ha un identificativo (run_id) e produce un report e un transcript. Se mancano dati per chiudere un trade o per valutare un segnale, il sistema non improvvisa: lo dichiara e lo contabilizza (forced exits, data_gaps).

7. Come nasce un ranking a stelle (in modo comprensibile)

Il ranking e pensato per essere leggibile: per ogni ticker si vuole rispondere a tre domande: **quanto** sembra promettente, **quanto** e affidabile la stima, e **perche** appare in lista.

7.1 Ingredienti del punteggio

- Storico (se disponibile):** come si sono comportate in passato combinazioni simili di fonte+rating (calibrazione).
- Sentiment:** un aggiustamento lieve basato sul testo della notizia (scala -1..+1).
- Controlli di eseguibilita:** un segnale entra in classifica solo se e 'enterable' (esiste una sessione D+1).

7.2 Formula semplificata (leggibile)

Nel codice (spec Wave 6) la stima di ritorno usa una componente storica 'shrinkata' e un piccolo aggiustamento di sentiment. In forma semplificata:

```
forecast_return_pct = shrunk_expected_return_pct + 0.20 * sentiment_score
confidence = min(1, n_bucket / 60)
Dove n_bucket e il numero di casi storici simili disponibili.
```

Le **stelle** sono percentili sul forecast (5 stelle = top 10%, 1 stella = bottom 10%), con tie-breaker deterministici.

7.3 Esempio reale nello snapshot (estratto)

rank	★	ticker	rating	forecast	conf.	headline (troncata)
1	★★★★★	AMD	BUY	0.50	0.00	Intel stock jumps 10% after CES reveal: are AI PCs next
2	★★★★★	AMZN	BUY	0.50	0.00	Mag 7 Stock Amazon Continues Its Winning Ways; Earn
3	★★★★★	C	BUY	0.50	0.00	TD Cowen Reaffirms Their Buy Rating on Spartan Delta
4	★★★★	NVDA	BUY	0.33	0.00	Citi reiterates Buy rating on Nvidia stock, maintains \$270
5	★★★★	AAPL	BUY	0.25	0.00	Apple Inc. \$AAPL Position Increased by Greenup Street
6	★★★★	AAPL	BUY	0.25	0.00	Apple Inc. \$AAPL Position Increased by Greenup Street

Nota: se non esistono ancora abbastanza trade storici per calibrare, la confidenza puo restare bassa e il ranking si appoggia soprattutto al sentiment; questo e dichiarato nei diagnostici dell'artefatto.

8. Cosa vede l'utente: la UI come 'cabina di pilotaggio'

La UI Streamlit e organizzata in pagine che corrispondono a fasi e ruoli. Non e un grafico fine a se stesso: e un pannello di controllo per evitare errori interpretativi.

Pagina UI	A chi serve	Cosa risponde (in parole semplici)
Decision Briefing	Operativo	Top pick del giorno, motivazioni, stato di fiducia e freschezza.
Pipeline Control	Operativo/Tech	Esegue migrate/tests/gates/run/certify e cattura transcript.
Gates & Data Quality	Operativo	Prima di fidarti: copertura prezzi, right-censoring, ticker mapping.
Audit Runs / Trades & Equity	Analisi	Risultati storici: run, ledger, curve e metriche.
Data Gaps & Backfill	Triage	Buchi dati e tentativi di riempimento (con reason code).
Forecasts & Ranking	Operativo	Classifica a stelle con breakdown e diagnostica.
NEWS-ALPHA	Operativo	Stato pipeline news, fixtures, history lane.
Lifecycle Monitor	Operativo	TRADABLE / WAITLIST / EXPIRED secondo AS-OF e TTL=0.

9. Come da questo documento derivano i canonici .doc

I canonici in **.doc** non sono testi indipendenti: sono una scomposizione 'per uso' di quanto descritto qui. Questo documento e il 'perche' e il 'che cosa'; i canonici sono il 'come', il 'dove' e il 'quando'.

Sezione di questo documento	Canonico derivato (owner)	Uso tipico
Visione, valore, perimetro	PROJ.md	Allineamento stakeholder; DR e scelte di governance.
Architettura e moduli (dettaglio)	TECH.md	Sviluppo e refactor; contratti tecnici.
Runbook operativo (comandi)	READ.md	Esecuzione quotidiana; procedure.
Dati (tabelle e semantica)	DDIC.md	Interrogazioni, troubleshooting, migrazioni.
Piano e backlog	WPLN.md + TODO.md	Roadmap e task; prioritizzazione.
Stato e storico	CURRENT_STATE.md + LOGBOOK.md + CHGLOG.md	Cosa e stato fatto; contesto decisionale.
Governance assistente	GUARDIAN_*.md	Regole per documentazione/prompt e riduzione drift.

10. Segmenti mancanti o parziali (Gap Register)

Durante il reverse engineering emergono aree dove il sistema è volutamente incompleto oppure dove esiste un placeholder. Renderlo visibile è parte del valore del progetto.

Area	Stato nello snapshot	Impatto e nota
Esecuzione ordini (broker)	Assente (fuori scope)	Non esiste trading live; la UI non deve essere interpretata come esecuzione
News real-time end-to-end	Parziale	Esistono collector e history lane; la parte real-time continuativa e fuori perimetro
Dating news (AS-OF vs event_date)	Parziale	History lane presente; contratto completo su asof_date/event_date va convalidato
Delivery/alert (notifiche)	Assente	Nessun sistema di alerting; potenziale add-on senza cambiare core.
Copertura corporate action	Parziale	Dividendi modellabili ma spesso disattivati; policy adjusted prices non attivati
Calibrazione statistica ricca	Dipende dai dati	Se audit_trades non contiene storico sufficiente, la confidenza del ranking è bassa

Raccomandazione

Prima di aggiungere nuove feature, è utile decidere che tipo di valore finale si vuole: (A) un terminale decisionale per operatore umano, (B) un framework di audit e ricerca, o (C) un sistema che evolva verso esecuzione. Ogni scelta cambia priorità e controlli.

Appendice A — Comandi essenziali (operatività)

Per coerenza multi-OS, nei documenti si usa **<PY>** come placeholder dell'interprete (Windows PowerShell tipico: `py -3.14`; Linux/macOS: `python`).

```
# Pipeline completa (offline-by-default) <PY> scripts/sentinel.py migrate <PY>
scripts/sentinel.py test <PY> scripts/sentinel.py verify <PY> scripts/sentinel.py
certify --offline # Ranking/forecast <PY> scripts/sentinel.py forecast # Avvio UI
(Streamlit) <PY> -m streamlit run app.py # NEWS-ALPHA (history lane GDELT) <PY>
scripts/news_alpha.py history download --allow-online --online --stream both
--date-from YYYY-MM-DD --date-to YYYY-MM-DD --with-gkgcounts <PY>
scripts/news_alpha.py history profile --stream events --date-from YYYY-MM-DD
--date-to YYYY-MM-DD
```

Appendice B — Glossario minimo

Termine	Spiegazione
DR (Decision Record)	Scheda che fissa una decisione architetturale/policy (cosa, perche, impatti) per evitare interpretazioni d
AS-OF	La data di riferimento della decisione: cosa posso sapere oggi senza usare dati futuri.
run_id	Identificativo di una esecuzione certificata. Permette di ricostruire input, output e log.
Gate	Controllo di qualita prima di lanciare l'audit (es. copertura prezzi, mapping ticker).
Forced exit	Uscita non 'naturale' di un trade per fine dati o problemi di copertura. Deve essere dichiarata.
Fixture	File deterministico (es. JSONL) che congela dati grezzi/normalizzati per analisi ripetibile.
DuckDB	Database embedded in un singolo file, facile da spostare e interrogare.

Appendice C — Inventory dei canonici .doc letti

File	Ruolo
CHLG.md	Changelog
CURRENT_STATE.md	Stato corrente
DDIC.md	Data dictionary (tabelle e semantica)
GUARDIAN_manual.md	Manuale guardian
GUARDIAN_rule.md	Regole di governance documentale
GUARDIAN_workflow.md	Workflow guardian
LOGBOOK.md	Log operativo e decisionale
PROJ.md	Overview + Decision Records (DR)
READ.md	Runbook operativo
TECH.md	Architettura tecnica e contratti
TODO.md	Backlog sintetico
WPLN.md	Workplan